Universität Regensburg

Fakultät für Sprach-, Literatur- und Kulturwissenschaften

Research Proposal

Im Studiengang Englische Sprachwissenschaft

Wintersemester 2025/26

Can AI Replace Traditional Textbooks?

A Comparative Corpus Analysis of LLM-Generated vs. Textbook Example Sentences

Autor:	Simon John
Matrikelnummer:	2175939
Adresse:	Wildbachweg 5, 93059 Regensburg
E-Mail:	simon.john@stud.uni-regensburg.de
Studiengang:	BA Englische Sprachwissenschaft
Abgabedatum:	February 13, 2026
Gutachter:	Dr. Thorsten Brato

Contents

1 Introduction **1**

2 Theoretical Framework **2**

2.1 Register Theory: Defining "Casualness" 2

2.1.1 Interactive Pronouns 2

2.1.2 Situation-Dependent Reference 3

2.1.3 Structural Reduction 3

2.2 Idiomaticity 3

2.2.1 The Open-Choice Principle 3

2.2.2 The Idiom Principle 4

2.3 Complexity 4

2.3.1 Krashen's Input Hypothesis 5

2.3.2 Sweller's Cognitive Load Theory 5

3 Methodology **6**

3.1 Corpus Compilation 6

3.1.1 Natural Reference Corpus (NRC) 6

3.1.2 Synthetic Control Corpus (SCC) 7

3.2 Computational Workflow 7

3.2.1 POS-Tagging 8

3.2.2 Quantifying Idiomaticity: LLM as a Judge 8

3.3 Statistical Analysis 8

4 Limitations **9**

4.1 Model Bias (Technical) 9

4.2 Proxy Measures (Methodological) 9

4.3 Model Self-Preference 10

4.4 Corpus Representativeness 10

5 Expected Outcomes and Alternative Hypotheses **11**

6 Conclusion **11**

References **13**

A TEC Source Textbooks (B1/B2 Subset) **16**

B Preliminary Results (SCC Pilot Study) **17**

B.1 Short Interpretation: *run* 18

B.2 Short Interpretation: *take* 18

C Python Code Documentation **20**
C.1 debug.py 20
C.2 scalable_context_generator.py 20
C.3 target_words.txt 23
C.4 sketch_engine_converter_nltk.py 24
C.5 pos_tagger.py 27
C.6 semantic_analyzer.py 36
C.7 visualizer.py 40

List of Figures

1 Aggregate proportion of `LITERAL` vs. `IDIOMATIC` usages by register (SCC pilot data). 17
2 Per-lemma heatmap of `IDIOMATIC` usage rates by register (SCC pilot data)..... 18

1 Introduction

Large Language Models (LLMs) like ChatGPT and Gemini are rapidly gaining recognition, even among the general public (see Mikhaylov et al. 2018). Due to their vast capabilities in content generation, automated evaluation and workflow optimisation, Artificial Intelligence is increasingly being introduced into various private and public sectors. One prominent domain is the language learning classroom, redirecting attention from static learning materials, such as textbooks and novels, to dynamic, interactive tools. However, while there already is plenty of research on how AI handles student output, such as automated essay scoring or error correction, there is a noticeable gap in how we view these models as input providers.

This gap is significant because traditional "Textbook English" has long been criticised for being too stiff and unnatural. Traditional learning material tends to be syntactically strict and overly explicit, presenting a "sanitised" version of English that rarely matches how people actually speak. Here is where generative AI offers a promising alternative: the potential to simulate "natural, casual" English on demand. The question, however, is whether LLMs can truly recreate the nuances of authentic speech, or if they simply produce a statistically average imitation that is just as stiff and artificial as the textbooks they aim to replace.

While this study is motivated by Krashen's Input Hypothesis, which argues that language acquisition depends on exposure to high-quality, comprehensible input, this paper pivots to a data-driven approach. By utilising Corpus Linguistics methods, this proposal addresses the following research question: *To what extent can Large Language Models (specifically Gemini Flash 2.5) generate linguistically distinct registers (formal, instructional, and casual) that quantitatively differ from human-authored textbook materials?*

To answer this question, this study compiles a Natural Reference Corpus (NRC) from existing B1/B2 textbook materials and contrasts it with a Synthetic Control Corpus (SCC) generated via role prompting across three register conditions: formal workplace language (HIGH register: questions and professional contexts), instructional language (NEUTRAL register: imperatives, directions and rules), and casual conversational language (LOW register: statements in informal contexts). This design enables testing whether textbook English aligns most closely with formal, instructional, or casual native registers, or whether it represents a distinct pedagogical register unlike any of these.

2 Theoretical Framework

To effectively compare synthetic example sentences against traditional textbook materials, we need to choose concrete linguistic criteria that go beyond vague impressions of "naturalness." This study grounds its analysis on the following criteria, building a robust framework that encapsulates three core areas of corpus linguistics: Register Theory, Idiomaticity and Complexity. Together, these criteria provide a basis that enables distinguishing between pedagogical language and authentic language usage.

2.1 Register Theory: Defining "Casualness"

As Biber (1988) argues, linguistic variation is never random and always functional. Linguistic features like pronouns, tense markers, or adverb classes tend to "co-occur" in texts, because they serve specific communicative functions (Biber 1988: 13). By conducting factor analysis on various corpora, Biber identified certain "dimensions" of variation. For this study, Dimension 1: Involved vs. Informational Production (Biber 1988: 107) is a critical concept that will help to quantify the casualness of a text.

Textbooks are typically designed to transmit rules and vocabulary explicitly to ensure understandability. This design leans heavily toward the "Informational Production" pole. Casual language, the target of many language learners, however, sits at the other end of the spectrum: "Involved Production". Therefore, we can define casualness not by the presence of slang, but by a high frequency of features associated with involved production.

If the AI effectively simulates a native casual register, the output text should be statistically marked by these three features:

2.1.1 Interactive Pronouns

The most obvious marker of involvement is the density of 1st and 2nd person pronouns (*I, me, we, you*). Biber notes that a high frequency of these pronouns indicates "interpersonal focus" (Biber 1988: 102). In contrast, textbooks tend to remove these interlocutors and focus on third-person objects or abstract nouns. *I* and *you* are much more frequent in conversation than other registers, because they act as an immediate anchor of discourse in direct interaction (Biber et al. 1999: 333).

2.1.2 Situation-Dependent Reference

Authentic conversations are rarely abstract. Most of the time, they refer to a specific time and place. This phenomenon is marked by adverbs of time and place like *here, now, tomorrow, then* or *yesterday*. Biber calls these "situation-dependent" references, because they rely strictly on the context for correct interpretation (Biber 1988: 37). Textbook sentences, on the one hand, are often constructed to be "timeless truths" (e.g. *"The bus leaves at 3 o'clock"*). Casual language, on the other hand, tends to be more immediate (e.g. *"When did you get here?"*).

2.1.3 Structural Reduction

Casualness can also be characterised by speed and economy of expression. In corpora of spontaneous conversation, this often surfaces as *structural reduction*: speakers compress forms that are predictable from context and rely on shared situational grounding. At the most visible level, this includes contractions (e.g. *can't* vs. *cannot*, *I'm* vs. *I am*), but reduction also extends to broader patterns such as complementiser deletion (e.g. omitting *that* in *"I hope you're well"*), ellipsis (dropping recoverable material in coordinated or adjacent clauses), and other shortened realisations that are common in spoken interaction (Biber et al. 1999).

For this proposal, structural reduction is relevant because textbook examples frequently avoid such reduced forms in favour of fully explicit, maximally transparent sentences. If the SCC-LOW is prompted to approximate a casual native register, it should therefore show a higher rate of these reductions (especially contractions and optional deletions) than the NRC, even when controlling for topic and sentence length.

2.2 Idiomaticity

2.2.1 The Open-Choice Principle

To quantify authenticity beyond register analysis, this study uses Sinclair's (1991) two frameworks of language processing. In the Open-Choice Principle, a sentence is viewed as a series of empty slots (e.g. Subject + Verb + Object) that has to be filled by respective words fulfilling the grammatical conditions. Textbooks heavily rely on this framework to demonstrate grammatical rules in a straightforward, understandable way. This, however, often results in sentences that are syntactically perfect, but idiomatically unnatural (e.g. *"I have entered the bus"*).

2.2.2 The Idiom Principle

A speaker of a language has a large number of preconstructed phrases of a specific language at their disposal. They then proceed to take these loose chunks and combine them to form a new, cohesive sentence. Native speakers tend to think in these chunks to form sentences, rather than performing Open-Choice (e.g. *"I got on the Bus"*).

This distinction is measurable by analyzing whether verbs are used literally (following the Open-Choice Principle) or idiomatically (following the Idiom Principle). Idiomatic usage includes phrasal verbs (e.g., *put up with* meaning *tolerate*), light-verb constructions (e.g., *take responsibility*), and metaphorical extensions (e.g., *run a company*). A statistically higher proportion of idiomatic usage in certain SCC registers would provide evidence that the AI can successfully mimic native phraseological patterns.

2.3 Complexity

A recurring issue in materials design is what one might call a "paradox of simplification": sentences are often simplified in order to help learners, but the result can on the contrary feel harder to process (Spiro et al. 1992). In this proposal, this is treated as a hypothesis about naturalness and learnability rather than an established claim. In particular, this study does not directly measure cognitive load. Instead, it explores whether corpus-based properties such as register, phraseology, and reduction correlate with differences in linguistic style that may relate to processing ease.

Accordingly, complexity is not treated as "long sentences" versus "short sentences," but as a question of how input is packaged and how much interpretive work is required to understand the text.

To make this more concrete, this study draws on two perspectives that are commonly mentioned in second language acquisition (SLA) and educational psychology, namely Krashen's Input Hypothesis (Krashen 1985) and Sweller's Cognitive Load Theory (Sweller 1988). These frameworks provide practical background needed to understand why differences in wording and information packaging might matter to the reader. They are not used to claim that any particular corpus feature (e.g. idiomatic density) reduces cognitive load.

2.3.1 Krashen's Input Hypothesis

Krashen's Input Hypothesis is often summarised with the formula $i + 1$. The basic idea is that acquisition depends on exposure to input that is largely comprehensible at the learner's current level (i), while still containing some new material the learner is unfamiliar with (+1) (Krashen 1985). This is most effectively achieved by immersion. For this study, this matters because the object of analysis is the example sentence. If sentences are the primary source of input in a textbook, then their register, phraseology and degree of contextualisation are not just stylistic features, but shape how easily learners can understand them.

At the same time, $i + 1$ highlights a problem that is relevant for AI-generated text. A sentence can be authentic and still be too dense for B1–B2 learners if it relies on too many interacting unfamiliar elements (idioms, reduced forms, implications). In contrast, a sentence can be perfectly transparent, but so controlled that it no longer resembles how English is actually used. In other words, the question is not simply whether AI can generate "native-like" output, but whether it can be pushed towards a plausible $i + 1$ zone for a given level.

2.3.2 Sweller's Cognitive Load Theory

Sweller's Cognitive Load Theory (1988) approaches the same problem from a different perspective. Sweller's hypothesis is based on the assumption that humans have a limited working memory, so learning is more effective when the material does not force learners to waste resources on avoidable difficulties. There is an important distinction between *intrinsic load*, which comes from the inherent complexity of the content, and *extraneous load*, which comes from how that content is presented. The presentation is particularly relevant for example sentences, because it is exactly where poorly chosen wording, weird information structure, or unnecessary redundancy can make an otherwise simple point harder than it needs to be.

In this proposal, CLT mainly functions as a motivation for focusing on presentation-related properties of sentences. However, this study does not test cognitive load directly. That is why any connection between observed patterns and processing effort is treated as suggestive rather than conclusive.

3 Methodology

This study uses a quantitative analysis based on corpora. The data collection and analysis are carried out with various Python scripts. For the AI-generated material, the scripts query *Gemini Flash 2.5* via its API. For tokenisation and part-of-speech tagging, they rely on the *Natural Language Toolkit* (NLTK) (Bird et al. 2009; Loper and Bird 2002). The goal is to keep the workflow transparent and easy to follow.

3.1 Corpus Compilation

This study aims to compare two different datasets: a Natural Reference Corpus (NRC) of authentic textbook English and a Synthetic Control Corpus (SCC) of LLM-generated sentences.

3.1.1 Natural Reference Corpus (NRC)

The NRC is compiled from the Textbook English Corpus (TEC), a specialised corpus assembled by Dr. Ellen Le Foll for investigating register variation in school EFL textbooks (Le Foll 2021b). The TEC is a dataset containing pedagogical materials from published ESL/EFL textbooks (Le Foll 2021a). Access to the TEC was granted through direct correspondence with the corpus compiler for academic research purposes.

Sampling procedure:

- CEFR filtering: A subcorpus was created containing only B1/B2 level materials to match the proficiency target of the SCC.
- Target lemma extraction: Concordance searches were performed for the same 10 high-frequency polysemous verbs used in the SCC (*take, make, hold, keep, break, run, drop, turn, see, look*).
- Verb-only filtering: All extracted sentences were processed through the same NLTK POS-tagging script described in Section 3.2.1 to collect only instances where the target lemma appears as a verb.

The NRC extraction and POS-filtering have been completed, yielding approximately 6200 sentences ready for semantic analysis. Upon approval, these sentences will be processed through the same `semantic_analyzer.py` workflow used for the SCC.

The TEC contains materials from 33 different ESL/EFL textbooks published between 2006 and 2018 by seven major publishers (Oxford University Press, Cambridge University Press, Klett, Cornelsen, Nathan, Bordas, Richmond). The corpus represents both European markets (German *Gymnasium* textbooks, French *collège/lycée* materials) and international coursebooks, providing a representative sample of most pedagogical English from the 2000s–2010s.

The TEC represents "traditional textbook English" as it appears in published pedagogical materials. It functions as the baseline for evaluating whether LLM-generated examples differ systematically in register and idiomaticity. Unlike general corpora (e.g. BNC, COCA), the TEC specifically captures the pedagogical register that this study aims to compare against AI-generated alternatives.

3.1.2 Synthetic Control Corpus (SCC)

The current workflow is a pilot study built around a controlled list of ten high-frequency, polysemous target lemmas (*take, make, hold, keep, break, run, drop, turn, see, look*). For each lemma, the script `scalable_context_generator.py` prompts Gemini Flash 2.5 via the API and automatically generates a set number of sentences under the following three conditions:

1. Register HIGH (Formal / Workplace; Questions)
2. Register NEUTRAL (Direct / Instructions; Imperatives)
3. Register LOW (Casual / Friends; Statements)

Each generated sentence is saved with metadata (`lemma`, `register`, `mood`) for register-based comparison.

3.2 Computational Workflow

Because textbook data is not structured and AI can produce occasional formatting mistakes, the processing follows a strict ETL workflow (Extract, Transform, Load) implemented in Python. In practice, this includes exporting the TEC sentence data from Sketch Engine and converting it into a consistent JSON/CSV format for further processing.

3.2.1 POS-Tagging

A central practical problem is that many of the target items are functionally ambiguous. For example, *run* can appear as a noun (NN) as well as a verb (VB). If we simply searched for strings, we would keep irrelevant tokens (e.g. *"a morning run"*) and distort the analysis.

To avoid this, the preprocessing step is handled by `pos_tagger.py`. The script tokenises each sentence and applies the NLTK Averaged Perceptron Tagger. It then lemmatises tokens until a matching verb occurrence is found, and keeps the sentence only if it finds an occurrence whose lemma matches the `lemma` field in the JSON file (case-insensitive) and is tagged as a verb (i.e. a tag starting with `V`, such as VB, VBD, VBG, VBN, VBP, VBZ). This lemma-based matching ensures that inflected verb forms (e.g. *runs*, *ran*, *running*) are counted as instances of the target lemma. Because the tagger operates on the same sentence JSON used throughout the workflow and performs basic NLTK checks before tagging, this step is reproducible and keeps the dataset consistent.

3.2.2 Quantifying Idiomaticity: LLM as a Judge

The current study analyses semantic behaviour directly by tagging each sentence-level occurrence of the target lemma as either `LITERAL` or `IDIOMATIC`. This is implemented as an *LLM-as-a-judge* step: instead of hand-coding every single step here (e.g. regex patterns for verb+particle sequences), Gemini is used to make a constrained semantic decision for each sentence (Zheng et al. 2023). This is appropriate here because idiomaticity is not limited to particle verbs. It is even more helpful, because many de-lexicalised or metaphorical uses (e.g. *run a company*) would be missed by standard regex rules.

This tagging is performed by `semantic_analyzer.py`, which sends small chunks of sentences to Gemini and requests a strict JSON array of labels. The dependent variable is therefore `Usage_Category` (`LITERAL` vs. `IDIOMATIC`), and the key grouping variable is `Register` (HIGH / NEUTRAL / LOW).

3.3 Statistical Analysis

The primary statistical test is a Chi-Square test of independence (χ^2) to check whether `Register` (HIGH / NEUTRAL / LOW) and `Usage_Category` (LITERAL / IDIOMATIC) are independent (Agresti 2012). The significance level is set to $\alpha = 0.05$.

Because the χ^2 test assumes sufficiently large frequencies, low counts in some lemma $\times$ register $\times$ category combinations may not be accurate. Therefore, Fisher's Exact Test (1992) will additionally be used as to check small expected frequencies.

The script `visualizer.py` additionally produces the two plots included in the appendix: an aggregate stacked bar chart by register and a per-lemma heatmap of `IDIOMATIC` rates.

4 Limitations

4.1 Model Bias (Technical)

This study uses Gemini 2.5 Flash for both data generation and semantic evaluation for API cost efficiency. While the model is great for high-volume processing, "Flash" models are optimised for latency rather than deep reasoning. That is why the model may show a "simplicity bias," potentially under-reporting complex idiomatic usage compared to larger reasoning models (e.g., Claude Opus 4.5 or GPT-5.2). Future iterations of this study could use a multi-LLM design to cross-validate results across different models.

4.2 Proxy Measures (Methodological)

Furthermore, the operationalisation of linguistic variables relies on proxy measures. For example, while Section 2.1.3 defines "structural reduction" briefly (including contractions and ellipsis), the current workflow quantifies it via a simple apostrophe-based contraction count. This count does not distinguish between clitics (e.g., "I'm") and possessive markers (e.g., "John's"), and it may also count apostrophes occurring inside quoted material. Ellipsis is not included in the present implementation.

The study couples register with grammatical mood (HIGH = Questions, NEUTRAL = Imperatives, LOW = Statements). This means that any observed differences in idiomaticity cannot be cleanly attributed to register (formal vs. casual) because they may instead be caused by mood-specific syntax (e.g. interrogatives often introduce auxiliaries such as *could* or *will* in subject-auxiliary inversion (Cambridge Dictionary n.d.)). A more robust design would decouple register and mood by crossing conditions (e.g. HIGH-Q workplace questions, HIGH-S workplace statements, LOW-Q casual questions, LOW-S casual statements). This would increase corpus size but

allow separating register effects from mood effects.

4.3 Model Self-Preference

A potential concern arises from the "LLM-as-a-judge" methodology. As the same model family (Gemini) is used to both generate the Synthetic Control Corpus and classify its semantic usage, there is a risk of self-preference bias. The model may be inclined to rate its own generations as contextually appropriate. Future research should employ an independent "judge" model to ensure independence during the evaluation process.

4.4 Corpus Representativeness

The NRC derived from an established pedagogical corpus (TEC), represents a specific subset of textbook English: materials explicitly designed for B1/B2 learners. This has two implications.

First, the findings may not generalise to other CEFR levels. A1/A2 materials may show even higher explicitness (lower idiomaticity), while C1/C2 materials may contain more authentic registers.

Second, the TEC compilation date range (2006–2018) means the NRC reflects textbook design practices from that period. Le Foll's (2021b) analysis of the TEC found that textbook English shows distinct register characteristics compared to general English corpora, particularly in its pedagogical framing and explicit grammatical structures. Therefore, if current ELT materials (2020–2025) show narrower gaps between textbook and authentic language, this would support H2 (Corpus-Informed Pedagogy Hypothesis) and suggest that LLMs are converging with an already-shifting policy of the design of learning material. On the other hand, if the 2006–2018 TEC materials already show substantial idiomaticity, this would challenge the assumption that textbooks avoid phraseological variation.

Despite these challenges, the B1/B2 focus is methodologically appropriate because this proficiency band represents the majority of adult ESL learners and is the primary target for intermediate textbooks where the difference between "teachability" and "authenticity" has arguably the most impact on the learner.

5 Expected Outcomes and Alternative Hypotheses

Based on the theoretical framework, three outcomes are possible:

H1 (Pedagogical Simplification Hypothesis): The NRC will show significantly higher `LITERAL` usage than all SCC registers, reflecting textbooks' reliance on the Open-Choice Principle (Sinclair 1991). If confirmed, this would suggest LLM-generated examples could complement textbooks by providing more idiomatic, register-appropriate input at the same CEFR level.

H2 (Corpus-Informed Pedagogy Hypothesis): The NRC will show pattern similar to SCC-NEUTRAL, indicating that modern textbooks already incorporate authentic phraseology. This would suggest that LLMs offer limited added value for idiomaticity.

H3 (Controlled Idiomaticity Hypothesis): The NRC will show selective idiomatic usage (e.g. high-frequency phrasal verbs such as *take off*) but avoid lower-frequency or colloquial idioms. This would indicate a pedagogical middle ground that neither purely literal examples nor fully native-like registers capture.

Preliminary χ^2 analysis (Section 3.3) will determine which hypothesis the data supports.

6 Conclusion

This proposal asks a practical question: can an LLM be pushed to generate example sentences that are not only well-formed, but also measurably different in register? Building on register variation research, the study compares LLM output to sentences already used in ESL/EFL textbooks and tests whether the distributions of semantic usage differ across conditions.

Rather than relying on intuition about what "sounds natural," the analysis treats each sentence as a unit and evaluates whether target-lemma patterns shift between `LITERAL` and `IDIOMATIC` realisations in ways consistent with existing frameworks of language use (Sinclair 1991). In practice, this matters because input quality is crucial to learnability. Register-appropriate, idiomatic but comprehensible examples are more likely to support acquisition without adding unnecessary processing load (Krashen 1985; Sweller 1988).

The NRC has been compiled from the TEC and is ready for analysis. Upon approval,

the NRC sentences will be semantically tagged using the validated SCC pipeline, and the resulting distributions will be compared against the SCC data to test the three competing hypotheses. Results will be reported in the final thesis.

References

Agresti, Alan (2012). *Categorical Data Analysis*. 3rd ed. Wiley.

Biber, Douglas (1988). *Variation across Speech and Writing*. 1st ed. Cambridge University Press. ISBN: 978-0-521-36543-7. DOI: 10.1017/CBO9780511621024.

Biber, Douglas, Stig Johansson, Geoffrey Leech, Susan Conrad, and Edward Finegan (1999). *Longman Grammar of Spoken and Written English*. Edinburgh: Pearson Education Ltd.

Bird, Steven, Ewan Klein, and Edward Loper (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. Sebastopol, CA: O'Reilly Media. ISBN: 9780596516499.

Cambridge Dictionary (n.d.). *Inversion*. URL: https://dictionary.cambridge.org/grammar/british-grammar/inversion (visited on 02/01/2026).

Fisher, Ronald A. (1992). "Statistical Methods for Research Workers". In: *Breakthroughs in Statistics: Methodology and Distribution*. Ed. by Samuel Kotz and Norman L. Johnson. New York, NY: Springer New York, pp. 66–70. ISBN: 978-1-4612-4380-9. DOI: 10.1007/978-1-4612-4380-9_6.

Krashen, Stephen D. (1985). *The Input Hypothesis: Issues and Implications*. London; New York: Longman. ISBN: 978-0-582-55381-1.

Le Foll, E. (2021a). *Bibliographic Metadata of the Textbook English Corpus (TEC). Textbook English: A Multi-Dimensional Approach, v. 1.1*. DOI: 10.5281/zenodo.4922819. (Visited on 01/31/2026).

Le Foll, E. (2021b). "Register variation in school EFL textbooks". *Register Studies* 3.2, pp. 207–246. DOI: 10.1075/rs.20009.lef. (Visited on 01/31/2026).

Loper, Edward and Steven Bird (2002). "NLTK: The Natural Language Toolkit". In: *Proceedings of the ACL-02 Workshop on Effective Tools and Methodologies for Teaching Natural Language Processing and Computational Linguistics*. Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, pp. 63–70. DOI: 10.3115/1118108.1118117.

Mikhaylov, Slava Jankin, Marc Esteve, and Averill Campion (Sept. 2018). "Artificial Intelligence for the Public Sector: Opportunities and Challenges of Cross-Sector Collaboration". *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*. ISSN: 1364-503X, 1471-2962. DOI: 10.1098/rsta.2017.0357. (Visited on 01/31/2026).

Sinclair, John McHardy (1991). *Corpus, Concordance, Collocation*. (Describing English Language). Oxford: Oxford university press. ISBN: 978-0-19-437144-5.

Spiro, Rand, Paul J. Feltovich, Michael Jacobson, and Richard Coulson (1992). "Cognitive Flexibility, Constructivism, and Hypertext: Random Access Instruction for Advanced Knowledge Acquisition in Ill-Structured Domains". *Constructivism and the technology of instruction: A conversation* 35, pp. 57–75.

Sweller, John (1988). "Cognitive Load During Problem Solving: Effects on Learning". *Cognitive Science* 12.2, pp. 257–285. ISSN: 1551-6709. DOI: 10.1207/s15516709cog1202_4. (Visited on 01/31/2026).

Zheng, Lianmin, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez,

and Ion Stoica (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. DOI: 10.48550/arXiv.2306.05685. (Visited on 01/31/2026).

Plagiarism Statement

I hereby declare that I have independently written the present work and have not used any aids other than those indicated. Any passages taken verbatim or in substance from other works (including those from electronic databases or the internet) have been clearly marked as such, with proper citation according to academic referencing rules. This declaration also applies to any visual representations, tables, maps, and drawings included in the work. I know any deception will be penalised according to the applicable study and examination regulations.

A Large Language Model (LLM) was used during the ideation process; however, the text and ideas presented are my own unless otherwise specified.

Regensburg, February 13, 2026

A TEC Source Textbooks (B1/B2 Subset)

The following textbook titles (B1/B2-level subset) were included in the TEC-derived NRC sample:

- *Achievers B1* (2015)
- *Achievers B1+* (2015)
- *Achievers B2* (2015)
- *Hi there!* 3e, Niveau A2–B1 (2015)
- *Join the Team* 4e, A2–B1 (2012)
- *Join the Team* 3e, A2–B1 (2013)
- *New Missions* 2de, A2–B1 (2014)
- *Solutions Pre-Intermediate* (2017)
- *Solutions Intermediate* (2016)
- *Solutions Intermediate Plus* (2017)

B Preliminary Results (SCC Pilot Study)

The following results are based on the AI-generated Synthetic Control Corpus (SCC) pilot data only. The Natural Reference Corpus (NRC) has been compiled and is ready for analysis. Semantic tagging and statistical comparison will be conducted upon approval.

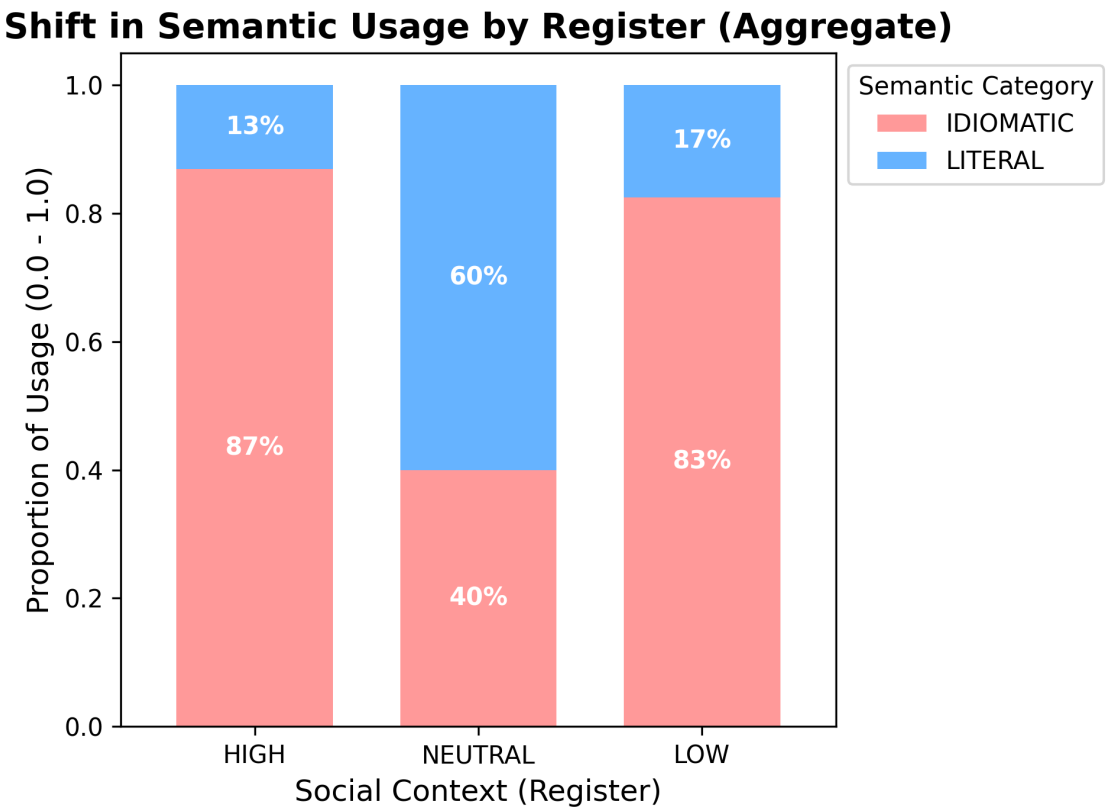

Figure 1: Aggregate proportion of `LITERAL` vs. `IDIOMATIC` usages by register (SCC pilot data).

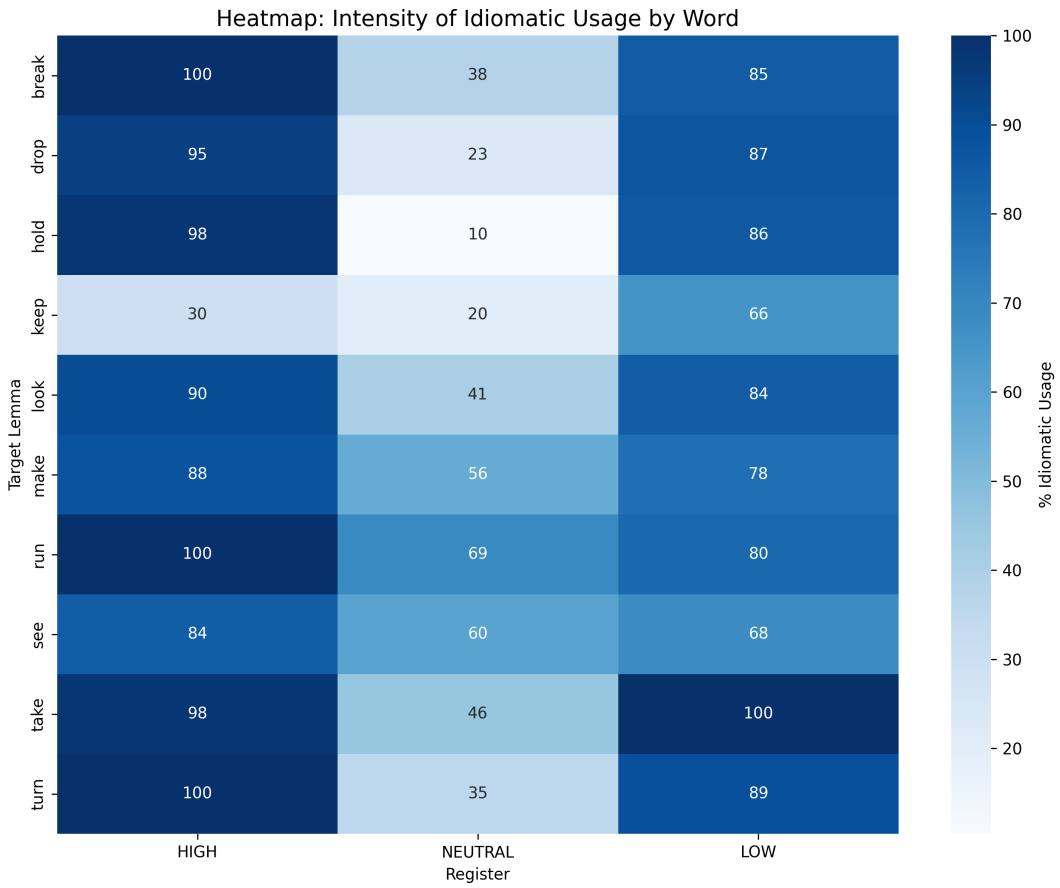

Figure 2: Per-lemma heatmap of IDIOMATIC usage rates by register (SCC pilot data).

B.1 Short Interpretation: *run*

In the pilot data, *run* is classified as 100% IDIOMATIC in the HIGH register. This aligns with the "Workplace/Professional" prompt condition: in professional contexts, *run* is typically used metaphorically (e.g. *run a meeting*, *run tests*, *run a project*) rather than to describe physical motion. For example, the synthetic HIGH-register sentence "Could you confirm if the new software will run efficiently on older systems?" uses *run* in the sense of "operate/function," i.e. a non-literal (de-lexicalised) usage.

In the LOW register, *run* is 80% IDIOMATIC. This is expected because casual contexts plausibly include literal movement (e.g. *I ran to the store*), increasing the share of LITERAL uses.

B.2 Short Interpretation: *take*

The word *take* shows a V-shaped pattern. In HIGH (formal register), 98% of uses are IDIOMATIC, light-verb constructions like *take responsibility* or *take measures*, where *take* carries minimal semantic weight (e.g. from the SCC: "Might this project take an unexpected turn given the new market analysis?"). In NEUTRAL (instructions), this

drops to 46%, with *take* used literally (*Take an exit*, *Take one tablet*). In LOW (casual), idiomaticity rises back to 100%, because of phrasal verbs like *take off* or *take it easy* (e.g. from the SCC: “She always takes her time getting ready, it’s so annoying.”).

In the SCC pilot, both formal and casual registers show high idiomaticity, while the AI-generated instructional register shows substantially lower idiomaticity. This pattern raises the question whether actual textbook-style examples similarly suppress idiomatic usage. This will be tested against the NRC in the full analysis.

C Python Code Documentation

Note: The code shown below uses simplified file paths for readability. The actual repository implementation uses organized directories (`outputs/`, `data/`, `src/`) and environment variables for API keys. See the GitHub repository for the actual code, dataset and version history.

`github.com/Eidolom/llm-corpus-generation-analysis`

C.1 debug.py

```
import google.generativeai as genai

# Paste key directly here for the test
API_KEY = "API_KEY_HERE"

genai.configure(api_key=API_KEY)
model = genai.GenerativeModel('gemini-2.5-flash')

print("--- TESTING GEMINI API ---")

try:
    response = model.generate_content("Write a sentence using the word 'run'.")
    print(f"Success! Response: {response.text}")
except Exception as e:
    print(f"Error: {e}")
```

C.2 scalable_context_generator.py

```
import google.generativeai as genai
import time
import json
import re

# --- CONFIGURATION ---
API_KEY = "API_KEY_HERE"
TARGET_WORDS_FILE = "target_words.txt"
OUTPUT_FILENAME = "intermediate_sentences.json"

# --- GENERATION SETTINGS ---
# 5 Batches x 10 sentences = 50 sentences per register (150 total per word)
NUM_BATCHES = 5
SENTENCES_PER_REG_PER_BATCH = 10
```

```
genai.configure(api_key=API_KEY)
model = genai.GenerativeModel('gemini-2.5-flash') # 2.5-flash is best for high-
    volume tasks

SYSTEM_PROMPT = """
You are a linguistic expert specialising in pragmatics and corpus generation.
Your task is to generate distinct, high-quality sentences for a target word
    based on strict register constraints.
The sentences must be appropriate for a CEFR B1/B2 learner.
"""

def clean_and_load_json(raw_text):
    """Clean markdown and parse JSON."""
    if not raw_text: return None
    try:
        # Regex to find the first [ and the last ]
        match = re.search(r'\[.*\]', raw_text, re.DOTALL)
        if match:
            return json.loads(match.group(0))
        else:
            # Fallback for simple strip
            cleaned = raw_text.replace('```json', '').replace('```', '').strip
    ()
            return json.loads(cleaned)
    except Exception as e:
        print(f"   JSON Parse Error: {e}")
        return None

def load_target_words(filename):
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            return [line.strip() for line in f if line.strip()]
    except FileNotFoundError:
        print(f" Error: '{filename}' not found.")
        return []

def generate_batch(word, batch_num):
    """Generates a single batch of 30 sentences (10 High, 10 Low, 10 Neutral)."
    ""

    prompt = f"""
    Target Word: "{word}"
```

```
    Batch ID: {batch_num} (Ensure these sentences are unique from generic
    examples)

    Generate a JSON array containing exactly {3 * SENTENCES_PER_REG_PER_BATCH}
    sentences:

    1. {SENTENCES_PER_REG_PER_BATCH} sentences: Register HIGH (Formal), Mood
    Question. Context: Workplace/Professional.
    2. {SENTENCES_PER_REG_PER_BATCH} sentences: Register LOW (Casual), Mood
    Statement. Context: Friends/Personal/Slang.
    3. {SENTENCES_PER_REG_PER_BATCH} sentences: Register NEUTRAL (Direct), Mood
     Imperative. Context: Instructions/Navigation/Rules.

    Vary the sentence structure and vocabulary usage.
    Output JSON keys: "register", "mood", "sentence".
    """

    try:
        response = model.generate_content(
            contents=[{"role": "user", "parts": [{"text": SYSTEM_PROMPT + "\n"
    + prompt}]}]
        )
        return clean_and_load_json(response.text)
    except Exception as e:
        print(f"   API Error: {e}")
        return None

def main():
    words = load_target_words(TARGET_WORDS_FILE)
    if not words: return

    all_data = []
    total_words = len(words)

    print(f"--- STARTING GENERATION: {total_words} Words x {NUM_BATCHES}
    Batches ---")
    print(f"Target: {NUM_BATCHES * SENTENCES_PER_REG_PER_BATCH} sentences per
    register per word.")

    for i, word in enumerate(words):
        word_sentences = []
        print(f"\n[{i+1}/{total_words}]  Processing '{word}'...")

        for batch_idx in range(NUM_BATCHES):
```

```
            print(f"   -> Generating Batch {batch_idx+1}/{NUM_BATCHES}...", end
    ="", flush=True)

            batch_data = generate_batch(word, batch_idx)

            if batch_data:
                # Validate we actually got a list
                if isinstance(batch_data, list):
                    # Add metadata
                    for item in batch_data:
                        item['lemma'] = word
                        item['Source'] = "Synthetic_V2"
                        item['CEFR_Target'] = "B1/B2"
                        # Normalize register names just in case
                        if 'register' in item:
                            item['register'] = item['register'].upper()

                    word_sentences.extend(batch_data)
                    print(f"  Got {len(batch_data)} sentences.")
                else:
                    print(f"  format error (not a list).")
            else:
                print("  Failed.")

            # Sleep slightly longer to avoid rate limits on the loop
            time.sleep(2.0)

        all_data.extend(word_sentences)
        print(f"    Total for '{word}': {len(word_sentences)} sentences.")

    # Save
    if all_data:
        with open(OUTPUT_FILENAME, 'w', encoding='utf-8') as f:
            json.dump(all_data, f, indent=4)
        print(f"\nSUCCESS! Generated {len(all_data)} total sentences.")
        print(f"Saved to {OUTPUT_FILENAME}")
    else:
        print("\n No data generated.")

if __name__ == "__main__":
    main()
```

C.3 target_words.txt

```
take
make
hold
keep
break
run
drop
turn
see
look
```

C.4 sketch_engine_converter_nltk.py

```python
import json
import re
import pandas as pd
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.tag import pos_tag

# --- NLTK SETUP ---
# NLTK requires downloading model files (tokenizers, taggers).
try:
    nltk.data.find('tokenizers/punkt_tab')
    nltk.data.find('taggers/averaged_perceptron_tagger_eng')
except LookupError:
    print("Downloading necessary NLTK models...")
    nltk.download('punkt_tab')
    nltk.download('averaged_perceptron_tagger_eng')

# --- CONFIGURATION ---
INPUT_FILE = 'textbook_sentences.json'
OUTPUT_JSON = 'semantic_analysis_results.json'
OUTPUT_CSV = 'semantic_analysis_summary.csv'

def clean_text(text):
    """Cleans messy textbook artifacts."""
    if not text: return ""
    # Remove double single quotes often found in SQL dumps
    text = text.replace("''", "'")
    # Remove excessive whitespace
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

```python
def get_context_window(words, index, window_size=5):
    """
    Since NLTK doesn't do full dependency parsing easily,
    we grab a 'context window' of words following the verb
    to capture the object/phrase (e.g., 'keep' -> 'in touch').
    """
    start = index + 1
    end = min(index + 1 + window_size, len(words))
    context_words = words[start:end]
    return " ".join(context_words)

def analyze_data(data):
    analyzed_rows = []
    print(f"Processing {len(data)} raw entries with NLTK...")

    for entry in data:
        raw_text = clean_text(entry.get('sentence', ''))
        target_lemma = entry.get('lemma', '').lower()

        # 1. Split chunk into sentences
        sentences = sent_tokenize(raw_text)

        for sent in sentences:
            # 2. Tokenize words
            words = word_tokenize(sent)

            # 3. Part-of-Speech Tagging
            # Returns list of tuples: [('Word', 'TAG'), ...]
            tagged_words = pos_tag(words)

            found_verb = False
            context_pattern = "N/A"

            for i, (word, tag) in enumerate(tagged_words):
                # Check if word matches target (normalized)
                # AND if tag starts with 'V' (VB, VBD, VBG, VBN, VBP, VBZ are
    all verbs)
                if word.lower() == target_lemma and tag.startswith('V'):
                    found_verb = True
                    # Grab the words following the verb
                    context_pattern = get_context_window(words, i)
                    break
```

```
            # 4. Filter and Save
            # We filter out very short fragments (e.g. headers)
            if found_verb and len(words) > 3:
                row = {
                    "Target_Lemma": target_lemma,
                    "Extracted_Sentence": sent,
                    "Context_Pattern": f"{target_lemma} + {context_pattern}",
                    "Original_Source": entry.get('Source', 'Unknown'),
                    "Register": entry.get('register', 'N/A')
                }
                analyzed_rows.append(row)

    return analyzed_rows

# --- MAIN EXECUTION ---
if __name__ == "__main__":
    print("--- STARTING SEMANTIC ANALYSIS (NLTK VERSION) ---")

    # 1. Load Data
    try:
        with open(INPUT_FILE, 'r', encoding='utf-8') as f:
            raw_data = json.load(f)
    except FileNotFoundError:
        print(f"Error: Could not find '{INPUT_FILE}'. Make sure it exists.")
        exit()

    # 2. Analyze
    results = analyze_data(raw_data)

    # 3. Save Results
    if results:
        with open(OUTPUT_JSON, 'w', encoding='utf-8') as f:
            json.dump(results, f, indent=4, ensure_ascii=False)

        df = pd.DataFrame(results)
        df.to_csv(OUTPUT_CSV, index=False)

        print(f"   Analysis Complete.")
        print(f"   Processed {len(raw_data)} chunks into {len(results)}
    specific sentences.")
        print(f"   Saved to '{OUTPUT_CSV}'.")
    else:
        print("No valid sentences containing the target verbs were found.")
```

C.5 pos_tagger.py

```
"""
Full-functioning POS (Part-of-Speech) Tagger
Loads sentences from JSON and performs comprehensive linguistic analysis
"""

import json
import nltk
from nltk import pos_tag, word_tokenize
from nltk.corpus import wordnet
from nltk.stem import WordNetLemmatizer
from nltk.chunk import ne_chunk
from pathlib import Path
from typing import List, Dict, Tuple, Any
import warnings

warnings.filterwarnings('ignore')

# Download required NLTK resources
required_resources = [
    'punkt',
    'averaged_perceptron_tagger',
    'wordnet',
    'maxent_ne_chunker',
    'words',
    'universal_tagset'
]

# Download each resource; use a simple find/download to avoid probing non-zip
    files
for resource in required_resources:
    try:
        nltk.data.find(resource)
    except LookupError:
        nltk.download(resource, quiet=True)


class POSTagger:
    """
    A comprehensive POS tagger for analyzing sentences with multiple linguistic
     features
    """

    def __init__(self, json_file: str = 'intermediate_sentences.json'):
```

```
    """
    Initialize the POS tagger and load sentences from JSON

    Args:
        json_file: Path to the JSON file containing sentences
    """
    self.json_file = json_file
    self.sentences_data = []
    self.lemmatizer = WordNetLemmatizer()
    self.pos_results = []

def load_sentences(self) -> bool:
    """
    Load sentences from the JSON file

    Returns:
        bool: True if successful, False otherwise
    """
    try:
        file_path = Path(self.json_file)

        if not file_path.exists():
            print(f"Error: File '{self.json_file}' not found")
            return False

        with open(file_path, 'r', encoding='utf-8') as f:
            self.sentences_data = json.load(f)

        print(f"Loaded {len(self.sentences_data)} sentences from '{self.
json_file}'")
        return True

    except json.JSONDecodeError as e:
        print(f"Error: Invalid JSON format - {e}")
        return False
    except Exception as e:
        print(f"Error: {e}")
        return False

def get_wordnet_pos(self, nltk_tag: str) -> str:
    """
    Convert NLTK POS tags to WordNet POS tags

    Args:
```

```
        nltk_tag: NLTK POS tag

    Returns:
        WordNet POS tag
    """
    if nltk_tag.startswith('J'):
        return wordnet.ADJ
    elif nltk_tag.startswith('V'):
        return wordnet.VERB
    elif nltk_tag.startswith('N'):
        return wordnet.NOUN
    elif nltk_tag.startswith('R'):
        return wordnet.ADV
    return None

def analyze_sentence(self, sentence: str, target_lemma: str) -> Dict[str,
Any]:
    """
    Modified to perform Thesis-Specific analysis (Section 3.2.1):
    1. Lemmatizes tokens FIRST to catch inflected forms (ran -> run).
    2. Checks if the lemma matches AND if it is a VERB.
    3. Extracts Context Window for the AI Judge.
    """
    # Tokenize and Tag
    tokens = word_tokenize(sentence)
    pos_tags = pos_tag(tokens)

    target_is_verb = False
    context_window = []
    lemmatized_tokens = []

    # 1. LEMMATIZATION LOOP
    for i, (token, tag) in enumerate(pos_tags):
        # Convert NLTK tag to WordNet tag for accurate lemmatization
        wn_pos = self.get_wordnet_pos(tag)
        if wn_pos:
            lemma = self.lemmatizer.lemmatize(token.lower(), pos=wn_pos)
        else:
            lemma = self.lemmatizer.lemmatize(token.lower())

        # Store for output
        lemmatized_tokens.append((token, tag, lemma))

        # 2. THE VERB FILTER (Crucial: Compare LEMMA, not TOKEN)
```

```
        # We look for the target lemma (e.g. 'run') matching the current
word's lemma (e.g. 'ran'->'run')
        # AND check if it has a Verb tag (VB, VBD, VBG, etc.)
        if lemma == target_lemma.lower() and tag.startswith('V'):
            target_is_verb = True

            # 3. THE CONTEXT WINDOW (For AI Judge/Idiom analysis)
            # Grab the next 5 tokens
            context_start = i + 1
            context_end = min(len(tokens), context_start + 5)
            context_window = tokens[context_start:context_end]

            # We stop after the first valid verb instance to keep the
context window specific
            break

    # 4. METRIC: CONTRACTION RATIO
    num_contractions = sum(1 for t in tokens if "'" in t)

    return {
        'tokens': tokens,
        'pos_tags': pos_tags,
        'lemmatized': lemmatized_tokens,
        'is_valid_verb': target_is_verb,  # <--- The Gatekeeper for the AI
Judge
        'context_window': context_window,
        'num_contractions': num_contractions,
        'token_count': len(tokens)
    }

def process_all_sentences(self) -> List[Dict[str, Any]]:
    if not self.sentences_data:
        print("No sentences loaded.")
        return []

    self.pos_results = []

    for idx, item in enumerate(self.sentences_data, 1):
        sentence = item.get('sentence', '')
        target_lemma = item.get('lemma', '') # Get the target word (e.g., '
run')

        if not sentence or not target_lemma:
            continue
```

```
        # Pass the target lemma to the analyzer
        analysis = self.analyze_sentence(sentence, target_lemma)

        # THESIS LOGIC: Only keep if it is actually used as a verb
        if analysis['is_valid_verb']:
            result = {
                'index': idx,
                'register': item.get('register', 'N/A'),
                'mood': item.get('mood', 'N/A'),
                'sentence': sentence,
                'target_lemma': target_lemma,
                'source': item.get('Source', 'N/A'),
                'cefr_level': item.get('CEFR_Target', 'N/A'),
                'analysis': analysis
            }
            self.pos_results.append(result)

    print(f"Processed and Filtered. Kept {len(self.pos_results)} valid verb
 sentences.")
    return self.pos_results

def display_analysis(self, max_items: int = 5) -> None:
    """
    Display POS analysis results in a readable format

    Args:
        max_items: Maximum number of sentences to display
    """
    if not self.pos_results:
        print("No results to display. Run process_all_sentences() first.")
        return

    for result in self.pos_results[:max_items]:
        print("\n" + "="*80)
        print(f"Index: {result['index']}")
        print(f"Register: {result['register']} | Mood: {result['mood']} |
CEFR: {result['cefr_level']}")
        print(f"Sentence: {result['sentence']}")
        print(f"Target Lemma: {result['target_lemma']}")
        print("\nPOS Tags:")
        for token, pos in result['analysis']['pos_tags']:
            print(f"  {token:15} -> {pos}")
        print("\nLemmatized Form:")
```

```
        for token, pos, lemma in result['analysis']['lemmatized']:
            print(f"  {token:15} ({pos:4}) -> {lemma}")

def get_statistics(self) -> Dict[str, Any]:
    """
    Get statistics about the processed sentences

    Returns:
        Dictionary with various statistics
    """
    if not self.pos_results:
        return {}

    # Count POS tags
    pos_tag_counts = {}
    total_tokens = 0

    for result in self.pos_results:
        for token, pos in result['analysis']['pos_tags']:
            pos_tag_counts[pos] = pos_tag_counts.get(pos, 0) + 1
            total_tokens += 1

    # Count registers and moods
    register_counts = {}
    mood_counts = {}
    cefr_counts = {}

    for result in self.pos_results:
        register = result['register']
        register_counts[register] = register_counts.get(register, 0) + 1

        mood = result['mood']
        mood_counts[mood] = mood_counts.get(mood, 0) + 1

        cefr = result['cefr_level']
        cefr_counts[cefr] = cefr_counts.get(cefr, 0) + 1

    return {
        'total_sentences': len(self.pos_results),
        'total_tokens': total_tokens,
        'avg_tokens_per_sentence': total_tokens / len(self.pos_results),
        'unique_pos_tags': len(pos_tag_counts),
        'pos_tag_distribution': dict(sorted(pos_tag_counts.items(), key=
lambda x: x[1], reverse=True)),
```

```
        'register_distribution': register_counts,
        'mood_distribution': mood_counts,
        'cefr_distribution': cefr_counts
    }

def display_statistics(self) -> None:
    """Display comprehensive statistics about the corpus"""
    stats = self.get_statistics()

    if not stats:
        print("No statistics available. Run process_all_sentences() first."
)
        return

    print("\n" + "="*80)
    print("CORPUS STATISTICS")
    print("="*80)
    print(f"Total Sentences: {stats['total_sentences']}")
    print(f"Total Tokens: {stats['total_tokens']}")
    print(f"Average Tokens per Sentence: {stats['avg_tokens_per_sentence
']:.2f}")
    print(f"Unique POS Tags: {stats['unique_pos_tags']}")

    print("\n--- POS Tag Distribution (Top 15) ---")
    for pos, count in list(stats['pos_tag_distribution'].items())[:15]:
        percentage = (count / stats['total_tokens']) * 100
        print(f"  {pos:6} : {count:6} ({percentage:5.2f}%)")

    print("\n--- Register Distribution ---")
    for register, count in sorted(stats['register_distribution'].items(),
key=lambda x: x[1], reverse=True):
        print(f"  {register:15} : {count:6}")

    print("\n--- Mood Distribution ---")
    for mood, count in sorted(stats['mood_distribution'].items(), key=
lambda x: x[1], reverse=True):
        print(f"  {mood:15} : {count:6}")

    print("\n--- CEFR Level Distribution ---")
    for cefr, count in sorted(stats['cefr_distribution'].items(), key=
lambda x: x[1], reverse=True):
        print(f"  {cefr:6} : {count:6}")
```

```
def export_results(self, output_file: str = 'pos_tagging_results.json') ->
bool:
    """
    Export POS tagging results to a JSON file

    Args:
        output_file: Path to the output JSON file

    Returns:
        bool: True if successful
    """
    try:
        # Convert analysis data to JSON-serializable format
        export_data = []
        for result in self.pos_results:
            export_result = {
                'index': result['index'],
                'register': result['register'],
                'mood': result['mood'],
                'sentence': result['sentence'],
                'target_lemma': result['target_lemma'],
                'source': result['source'],
                'cefr_level': result['cefr_level'],
                'analysis': {
                    'tokens': result['analysis']['tokens'],
                    'pos_tags': [{'token': t, 'pos': p} for t, p in result[
'analysis']['pos_tags']],
                    'lemmatized': [{'token': t, 'pos': p, 'lemma': l} for t
, p, l in result['analysis']['lemmatized']],
                    'token_count': result['analysis']['token_count']
                }
            }
            export_data.append(export_result)

        with open(output_file, 'w', encoding='utf-8') as f:
            json.dump(export_data, f, indent=2, ensure_ascii=False)

        print(f"Results exported to '{output_file}'")
        return True
    except Exception as e:
        print(f"Error exporting results: {e}")
        return False

def get_lemma_occurrences(self, target_lemma: str) -> List[Dict[str, Any]]:
```

```
        """
        Find all occurrences of a specific lemma in the corpus

        Args:
            target_lemma: The lemma to search for

        Returns:
            List of sentences containing the lemma
        """
        occurrences = []

        for result in self.pos_results:
            lemmatized = result['analysis']['lemmatized']
            for token, pos, lemma in lemmatized:
                if lemma.lower() == target_lemma.lower():
                    occurrences.append({
                        'sentence': result['sentence'],
                        'register': result['register'],
                        'mood': result['mood'],
                        'cefr_level': result['cefr_level'],
                        'pos_tag': pos,
                        'token': token
                    })
                    break

        return occurrences


def main():
    """Main execution function"""
    print("POS TAGGER - Loading and Processing Sentences")
    print("="*80)

    # Initialize tagger
    tagger = POSTagger('intermediate_sentences.json')

    # Load sentences
    if not tagger.load_sentences():
        return

    # Process all sentences
    tagger.process_all_sentences()

    # Display sample results
```

```
    print("\n--- SAMPLE RESULTS (First 3 Sentences) ---")
    tagger.display_analysis(max_items=3)

    # Display statistics
    tagger.display_statistics()

    # Export results
    tagger.export_results('pos_tagging_results.json')

    # Example: Find a specific lemma
    print("\n" + "="*80)
    print("SEARCHING FOR LEMMA 'take'")
    print("="*80)
    take_occurrences = tagger.get_lemma_occurrences('take')
    print(f"Found {len(take_occurrences)} occurrences of 'take'")

    for i, occurrence in enumerate(take_occurrences[:5], 1):
        print(f"\n{i}. {occurrence['sentence']}")
        print(f"   Token: {occurrence['token']} | POS: {occurrence['pos_tag']}
    | Register: {occurrence['register']}")


if __name__ == '__main__':
    main()
```

C.6 semantic_analyzer.py

```
import google.generativeai as genai
import pandas as pd
import time
import json
import re
import math

# --- CONFIGURATION ---
API_KEY = "API_KEY_HERE"
INPUT_FILENAME = "intermediate_sentences.json"
OUTPUT_FILENAME = "thesis_semantic_data_final.csv"

# BATCH SIZE FOR ANALYSIS
# We will process 20 sentences at a time. This is the "Safe Zone" for LLM
    counting.
CHUNK_SIZE = 20
```

```
genai.configure(api_key=API_KEY)
model = genai.GenerativeModel('gemini-2.5-flash')

SYSTEM_PROMPT = """
You are a semantic analyzer. I will provide a list of sentences containing a
    target word.
Classify the usage of the target word in each sentence into exactly one
    category:
1. "LITERAL" (Physical/Core meaning).
2. "IDIOMATIC" (De-lexicalized, Metaphor, Phrasal Verb, or Fixed Phrase).

Return a strict JSON ARRAY of strings.
Example Output: ["LITERAL", "IDIOMATIC", "LITERAL"]
"""

def load_sentences(filename):
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            return json.load(f)
    except Exception as e:
        print(f" Error loading input file: {e}")
        return None

def get_tags_for_chunk(word, chunk_sentences, retry_count=0):
    """
    Analyzes a small chunk of sentences.
    """
    prompt = f"""
    Target Word: "{word}"
    Sentences to classify ({len(chunk_sentences)} items):
    {json.dumps(chunk_sentences)}

    Return exactly {len(chunk_sentences)} tags in a JSON array.
    """

    try:
        response = model.generate_content(
            contents=[{"role": "user", "parts": [{"text": SYSTEM_PROMPT + "\n"
    + prompt}]}]
        )

        raw_text = response.text.strip()

        # Cleaner Regex to find JSON array
```

```
        match = re.search(r'\[.*?\]', raw_text, re.DOTALL)
        if not match:
            # If regex fails, try parsing raw text if it looks like a list
            if raw_text.startswith('[') and raw_text.endswith(']'):
                json_str = raw_text
            else:
                raise ValueError("No JSON array found")
        else:
            json_str = match.group(0)

        tags = json.loads(json_str)

        # Validate Count
        if len(tags) != len(chunk_sentences):
            # Retry once if count is wrong
            if retry_count < 1:
                print(f"     Count mismatch ({len(tags)} vs {len(
    chunk_sentences)}). Retrying...")
                time.sleep(1)
                return get_tags_for_chunk(word, chunk_sentences, retry_count=1)
            else:
                print(f"     Failed chunk for '{word}'. Expected {len(
    chunk_sentences)}, got {len(tags)}.")
                return ["ERROR"] * len(chunk_sentences)

        return [str(t).upper() for t in tags]

    except Exception as e:
        print(f"     Error: {e}")
        return ["ERROR"] * len(chunk_sentences)

def main():
    data = load_sentences(INPUT_FILENAME)
    if not data: return

    # Group by Lemma
    grouped_data = {}
    for item in data:
        lemma = item['lemma']
        if lemma not in grouped_data: grouped_data[lemma] = []
        grouped_data[lemma].append(item)

    final_rows = []
    sorted_words = sorted(grouped_data.keys())
```

```
total_words = len(sorted_words)

print(f"--- STARTING CHUNKED ANALYSIS FOR {total_words} WORDS ---")

for i, word in enumerate(sorted_words):
    records = grouped_data[word]
    total_items = len(records)
    print(f"[{i+1}/{total_words}]  Analyzing '{word}' ({total_items}
sentences)...")

    # --- THE CHUNKING LOOP ---
    word_tags = []
    num_chunks = math.ceil(total_items / CHUNK_SIZE)

    for batch_idx in range(num_chunks):
        start = batch_idx * CHUNK_SIZE
        end = start + CHUNK_SIZE
        chunk_records = records[start:end]
        chunk_sentences = [r['sentence'] for r in chunk_records]

        # Get tags for just this small batch
        tags = get_tags_for_chunk(word, chunk_sentences)
        word_tags.extend(tags)

        # print(f"    Batch {batch_idx+1}/{num_chunks}: Processed {len(tags
)} items.")
        time.sleep(0.5) # Fast sleep for small chunks

    # Merge tags back
    for record, tag in zip(records, word_tags):
        row = {
            "Lemma": record['lemma'],
            "Register": record['register'],
            "Mood": record['mood'],
            "Usage_Category": tag,
            "Full_Sentence": record['sentence']
        }
        final_rows.append(row)

# Save
df = pd.DataFrame(final_rows)
df.to_csv(OUTPUT_FILENAME, index=False)

print("\n" + "="*40)
```

```
    print(f" DONE. Saved to {OUTPUT_FILENAME}")
    print("Summary of Results:")
    print(df['Usage_Category'].value_counts())
    print("="*40)

if __name__ == "__main__":
    main()
```

C.7 visualizer.py

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import chi2_contingency

# --- CONFIGURATION ---
INPUT_FILENAME = "thesis_semantic_data_final.csv"

# 1. Load Data
df = pd.read_csv(INPUT_FILENAME)

# Clean up Register names just in case (ensure UPPERCASE)
df['Register'] = df['Register'].str.upper()

# Define specific order for charts (Logical progression)
register_order = ['HIGH', 'NEUTRAL', 'LOW']

# --- PART 1: THE MAIN FINDING (Stacked Bar Chart) ---
print("\n--- 1. GENERATING AGGREGATE CHART ---")

# Calculate counts and percentages
cross_tab = pd.crosstab(df['Register'], df['Usage_Category'])
cross_tab = cross_tab.reindex(register_order)
cross_tab_prop = cross_tab.div(cross_tab.sum(1), axis=0)

# Plotting
plt.figure(figsize=(10, 6))
# Plot the bars (stacked)
ax = cross_tab_prop.plot(kind='bar', stacked=True, color=['#ff9999', '#66b3ff'
    ], width=0.7)

# Formatting
plt.title('Shift in Semantic Usage by Register (Aggregate)', fontsize=14,
    fontweight='bold')
```

```
plt.ylabel('Proportion of Usage (0.0 - 1.0)', fontsize=12)
plt.xlabel('Social Context (Register)', fontsize=12)
plt.xticks(rotation=0)
plt.legend(title='Semantic Category', loc='upper left', bbox_to_anchor=(1, 1))

# Add percentage labels on the bars
for c in ax.containers:
    ax.bar_label(c, fmt='%.0f%%', label_type='center', color='white',
    fontweight='bold')

plt.tight_layout()
plt.savefig('results_aggregate_chart.png', dpi=300)
print(" Saved 'results_aggregate_chart.png'")

# --- PART 2: THE STATISTICAL TEST ---
print("\n--- 2. RUNNING CHI-SQUARE TEST ---")
# We test if Register and Usage are independent
chi2, p, dof, expected = chi2_contingency(cross_tab)

print(f"Chi-Square Statistic: {chi2:.4f}")
print(f"P-Value: {p:.4e}")
if p < 0.05:
    print(">> RESULT: Statistically Significant! (You can reject the Null
    Hypothesis)")
else:
    print(">> RESULT: Not Significant.")

# --- PART 3: THE WORD BREAKDOWN (Heatmap) ---
print("\n--- 3. GENERATING WORD HEATMAP ---")

# Calculate % Idiomatic for every word in every register
pivot = df.groupby(['Lemma', 'Register'])['Usage_Category'].value_counts(
    normalize=True).unstack(fill_value=0)

# We only care about the "IDIOMATIC" column for the heatmap
if 'IDIOMATIC' in pivot.columns:
    heatmap_data = pivot['IDIOMATIC'].unstack().reindex(columns=register_order)

    plt.figure(figsize=(10, 8))
    sns.heatmap(heatmap_data, annot=True, cmap="Blues", fmt=".0%", cbar_kws={'
    label': '% Idiomatic Usage'})
    plt.title('Heatmap: Intensity of Idiomatic Usage by Word', fontsize=14)
    plt.ylabel('Target Lemma')
    plt.xlabel('Register')
```

```
    plt.tight_layout()
    plt.savefig('results_heatmap.png', dpi=300)
    print("Saved 'results_heatmap.png'")
else:
    print("Could not generate heatmap (Label 'IDIOMATIC' missing?)")

print("\nDONE. Open the PNG files to see your thesis results.")
```